COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/05/2026	**DATE SUBMITTED:**	08/05/2026
NAME:	Jiro Luis A. Katindig	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **Queue**

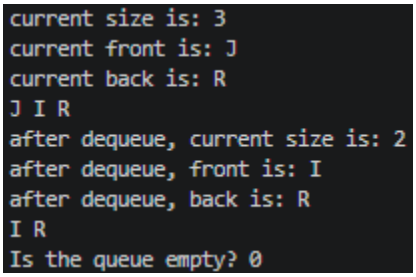
1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Whole code for ILO A

```cpp
#include<iostream>
#include<queue>

void display(std::queue<char> copyQ);
int main(){

    //create an object:
    std::queue<char> myQ;

    //use the enqueue operation
    myQ.push('J');
    myQ.push('I');
    myQ.push('R');
    std::cout<<"current size is: "<< myQ.size()<<std::endl;
    std::cout<<"current front is: "<< myQ.front()<<std::endl;
    std::cout<<"current back is: "<< myQ.back()<<std::endl;
    display(myQ);
    //use the dequeue operation
    myQ.pop();
    std::cout<<"after dequeue, current size is: "<< myQ.size()<<std::endl;
    std::cout<<"after dequeue, front is: "<< myQ.front()<<std::endl;
    std::cout<<"after dequeue, back is: "<< myQ.back()<<std::endl;


    display(myQ);

    //check the queue if it is empty
    std::cout<<"Is the queue empty? " << myQ.empty() <<std::endl;

    return 0;
}

//note noly use the member functions of the queue STL
void display(std::queue<char> copyQ){
    //create a copy of the queue
    std::queue<char> temp = copyQ;

    // loop until empty
    while(!temp.empty()){
        // display the front
        std::cout << temp.front() << " ";
        // dequeue the front
        temp.pop();
    }
    // add a new line
    std::cout << std::endl;
}
```

This code uses C++'s built-in std::queue instead of making one from scratch. In main, it creates a queue, pushes in 'J', 'I', 'R' to enqueue them, then prints the size, front, and back. There's also a display function that prints all the items by copying the queue first, then popping from the copy until it's empty, since you can't just loop through a queue directly in C++. After that, it pops once to dequeue, prints the size/front/back again, displays the queue again, then checks if it's empty.

Output of ILO A


The output matches the code perfectly. After pushing J, I, R, the size is 3, front is J, back is R, and it displays as "J I R". After popping once, J is removed, so size becomes 2, front is now I, back is still R, and it displays "I R". Lastly, it prints "Is the queue empty? 0" because the queue still has items, and since empty() returns true or false, it just shows up as 0 for false instead of a word.

COMPUTER ENGINEERING

QUEZON CITY

ILO B.1. Linked List Implementation

Whole code for ILO B.1

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"Testing the enqueue operator: \n";
    std::cout<<"enqueueing the following items: J, I, R, O\n";
    enqueue('J', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    enqueue('I', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    enqueue('R', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    enqueue('O', &front, &back);
    std::cout<<"front: "<< front->data << " "<<"back: "<<back->data <<std::endl;

    std::cout<< std::endl;
    std::cout<<"Testing the display all: \n";
    displayAll(front);
    std::cout<< std::endl;
    std::cout<<"Testing the isEmpty operator: \n";
    if(isEmpty(front, back)){
        std::cout<<"queue is empty!"<<std::endl;
    } else {
        std::cout<<"queue is not empty!"<<std::endl;
    }
    std::cout<< std::endl;

    std::cout<<"Testing the dequeue operator: \n";
    dequeue(&front, &back);
    if(!isEmpty(front, back)){
        std::cout<<"dequeueing the front, current front is: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    } else {
        std::cout<<"dequeueing the front, queue is now empty."<<std::endl;
    }
    dequeue(&front, &back);
    if(!isEmpty(front, back)){
        std::cout<<"dequeueing the front, current front is: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    } else {
        std::cout<<"dequeueing the front, queue is now empty."<<std::endl;
    }

    std::cout<< std::endl;
    std::cout<<"Testing the display all: \n";
    displayAll(front);

    std::cout<< std::endl;
    std::cout<<"Testing the isEmpty operator: \n";
    if(isEmpty(front, back)){
        std::cout<<"queue is empty!"<<std::endl;
    } else {
        std::cout<<"queue is not empty!"<<std::endl;
    }

    std::cout<< std::endl;
    dequeue(&front, &back);
    if(!isEmpty(front, back)){
        std::cout<<"dequeueing the front, current front is: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    } else {
        std::cout<<"dequeueing the front, queue is now empty."<<std::endl;
    }
    dequeue(&front, &back);
    if(!isEmpty(front, back)){
        std::cout<<"dequeueing the front, current front is: "<< front->data << " "<<"back: "<<back->data <<std::endl;
    } else {
        std::cout<<"dequeueing the front, queue is now empty."<<std::endl;
    }

    std::cout<< std::endl;
    std::cout<<"Testing the display all: \n";
    displayAll(front);

    std::cout<< std::endl;
    std::cout<<"Testing the isEmpty operator: \n";
    if(isEmpty(front, back)){
        std::cout<<"queue is empty!"<<std::endl;
    } else {
        std::cout<<"queue is not empty!"<<std::endl;
    }

    return 0;
}
```

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};

// new node creator
template <typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;

    //store the newdata to the newNode
    newNode ->data = newData;

    //point the newNode next to null
    newNode ->next = nullptr;

    return newNode;

}

// enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr ){
    //create a new node
    qNode<T>* newNode = new_node(newData);

    //check if the queue is empty
    if((*frontPtr)==nullptr && (*backPtr)== nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
    // if not then insert at the back
    (*backPtr)->next = newNode;
    (*backPtr)= newNode;

}
```

```cpp
// dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){

    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<< "the queue is empty"<<std::endl;
        return;
    }
    // create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to the deleteTemp
    deleteTemp = (*frontPtr);

    // check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;

    // delete na node
    delete deleteTemp;
}
```

```cpp
// display all
template <typename T>
void displayAll(qNode<T>* frontPtr){
    //check if the queue is empty
    if(frontPtr == nullptr){
        std::cout<<"the queue is empty"<<std::endl;
        return;
    }

    //create a temp pointer to walk through the queue
    qNode<T>* temp = frontPtr;

    //loop until temp reaches the end
    while(temp != nullptr){
        std::cout<< temp->data << " ";
        temp = temp->next;
    }
    std::cout<<std::endl;
}

// isEmpty
template <typename T>
bool isEmpty(qNode<T>* frontPtr, qNode<T>* backPtr){
    return (frontPtr == nullptr && backPtr == nullptr);
}

#endif
```

For this one, I made a queue from scratch using a linked list instead of the built-in one. I wrote enqueue, dequeue, displayAll, and isEmpty in queueLL.h. Enqueue adds a new node at the back, or sets it as both front and back if the queue was empty. Dequeue removes the front node, with a special check for when only one item is left so front and back both get set to nullptr properly. DisplayAll prints all items from front to end, and isEmpty checks if front and back are both nullptr. In main, I enqueue J, I, R, O, print front/back each time, display the queue, check if it's empty, then dequeue a few times with safety checks so it won't crash

Output of ILO B.1

```
Testing the enqueue operator:
enqueueing the following items: J, I, R, O
front: J back: J
front: J back: I
front: J back: R
front: J back: O

Testing the display all:
J I R O

Testing the isEmpty operator:
queue is not empty!

Testing the dequeue operator:
dequeueing the front, current front is: I back: O
dequeueing the front, current front is: R back: O

Testing the display all:
R O

Testing the isEmpty operator:
queue is not empty!

dequeueing the front, current front is: O back: O
dequeueing the front, queue is now empty.

Testing the display all:
the queue is empty

Testing the isEmpty operator:
queue is empty!
```

This is the actual output when I ran it, and it all matches what should happen. After enqueueing J, I, R, O, front stays J and back updates each time, ending at O. Displaying shows "J I R O", and isEmpty says not empty, which is correct. After dequeueing twice, front moves from I to R, back stays O. Displaying shows "R O", still not empty. After two more dequeues, front becomes O then the queue empties out, printing "queue is now empty" instead of crashing thanks to the safety check. The final display shows "the queue is empty" and isEmpty confirms "queue is empty!", so everything checks out.

ILO B.2. Array-Based Implementation

Whole code for ILO B.2

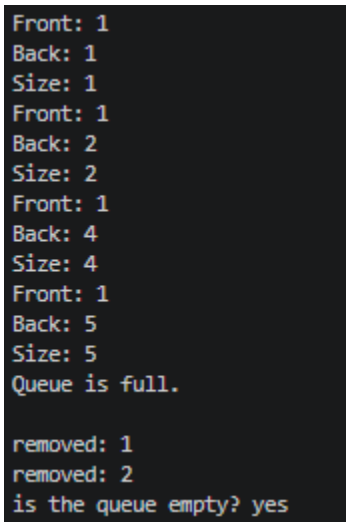
```cpp
#include <iostream>
#include "queuearr.h"

int main()
{
    // creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.enqueue(1);

    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(2);

    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(3);

    q.enqueue(4);

    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(5);

    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(6);

    std::cout << "\nremoved: " << q.dequeue() << std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;

    // testing the clear
    q.Clear();

    std::cout << "is the queue empty? "
              << (q.Empty() ? "yes" : "no")
              << std::endl;

    return 0;
}
```

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

//array based circular queue
template <typename T>
class queueArr{
        private:
        //pointer to dynamically allocate array
        T* q_array;
        //maximum number of elements a queue can hold
        size_t q_capacity;
        //current number of the elements in the queue
        size_t q_size;
        //index of the front elemet
        int q_front;
        //index of the rear element
        int q_back;
    public:
        //constructor
        queueArr(size_t capacity = 10);
        //copy constructor
        queueArr(const queueArr& other);
        //copy assignment operator
        queueArr& operator=(const queueArr& other);
        //destructor
        ~queueArr();
        //queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T front();
        T back();
        void enqueue(T value);
        T dequeue();
};
//Constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    //set capacity
    q_capacity = capacity;
    //allocated the array
    q_array = new T[q_capacity];
    q_front = 0;
    q_back = -1;
    q_size = 0;
}
//Destructor
template<typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}

//Empty
template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

//Full
template <typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;
}
```

```cpp
//Size
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

//Front
template <typename T>
T queueArr<T>::front(){
    //check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty.\n";
        return T();
    }
    //return
    return q_array[q_front];
}

//Back
template <typename T>
T queueArr<T>::back(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
        return T();
    }
    //return
    return q_array[q_back];
}

//Clear
template <typename T>
void queueArr<T>::Clear(){
    //reset the q_size, q_front, q_back;
    q_front = 0;
    q_back = -1;
    q_size = 0;
}

//Enqueue
template <typename T>
void queueArr<T>::enqueue(T value){
    //check if the queue is full
    if(Full()){
        std::cout<<"Queue is full."<<std::endl;
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    //store the value to the back
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
}

//Dequeue
template <typename T>
T queueArr<T>::dequeue(){
    //check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty."<<std::endl;
        return T();
    }
    //create a temporary variable to store the current front
    T temp = q_array[q_front];
    ///move q_front circularly
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    //reset the indexes if the queue become empty:
    if(Empty()){
        q_front = 0;
        q_back = -1;
    }
    //return temporary variable
    return temp;
}
#endif
```

For this one, I made a queue using a circular array instead of a linked list. In queuearr.h, I built a queueArr class with q_front, q_back, and q_size to track the queue. Enqueue checks if it's full, then adds the value and moves q_back forward using modulo so it wraps around, making it circular. Dequeue checks if it's empty, grabs the front value, then moves q_front forward the same way. I also added Empty, Full, Size, Clear, front, and back functions. In main, I make a queue with capacity 5, enqueue 10 to 50 one by one while printing front, back, and size, dequeue twice, clear the queue, then check if it's empty.

Output of ILO B.2



This is the actual output when I ran it, and it matches the code. After enqueueing 1, front is 1, back is 1, size is 1. After enqueueing 2, back becomes 2, size becomes 2. After enqueueing 3 and 4, back is 4 and size is 4, front stays 1 since nothing's been removed yet. After enqueueing 5, back becomes 5, size becomes 5, which fills up the capacity of 5. Trying to enqueue one more prints "Queue is full," which shows the full check works. Then I dequeue twice, removing 1 and 2, which prints "removed: 1" and "removed: 2". After that I call Clear(), and checking if the queue is empty prints "yes", which confirms Clear() resets everything properly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using queue implementation

ILOC.cpp

```cpp
#include <iostream>
#include "job.h"
#include "printer.h"

int main(){

    // create the printer object
    Printer officePrinter;
    std::cout << std::endl;

    std::cout << "Simulating multiple users sending print jobs...\n" << std::endl;

    // simulate different users sending jobs to the shared printer
    officePrinter.addJob(Job("Jiro", 5));
    officePrinter.addJob(Job("Luis", 12));
    officePrinter.addJob(Job("Dennise", 3));
    officePrinter.addJob(Job("Anne", 7));
    officePrinter.addJob(Job("Rochelle", 10));

    std::cout << std::endl;
    officePrinter.showStatus();
    std::cout << std::endl;

    std::cout << "Printer will now process the jobs in order (FCFS):\n" << std::endl;
    officePrinter.processAllJobs();

    officePrinter.showStatus();
    std::cout << std::endl;

    // try printing again even though the queue is already empty
    std::cout << "Trying to process another job after the queue is empty:" << std::endl;
    officePrinter.processNextJob();

    std::cout << std::endl;
    std::cout << "Is the printer queue empty now? "
              << (officePrinter.isEmpty() ? "yes" : "no") << std::endl;

    return 0;
}
```

For this one, I simulated a shared printer using a queue I built myself with a linked list, no STL. I made a Job class to hold each job's info, and a Printer class that handles adding jobs, assigning IDs automatically, and processing them one by one in the order they came in. I also added a status check to see how many jobs are left at any point, and a way to check if the queue is empty. In main, I simulated five different users sending print jobs one after another, checked the status, processed everything in order, checked the status again, then tried processing one more time on the already empty queue just to make sure it handles that properly instead of crashing.

Job.h

```h
#ifndef JOB_H
#define JOB_H

#include <string>

// class Job: represents one print job
class Job {
    public:
        int id;
        std::string userName;
        int numPages;

        // default constructor
        Job() : id(0), userName(""), numPages(0) {}

        // parameterized constructor, id gets assigned later by the Printer
        Job(std::string user, int pages)
            : id(0), userName(user), numPages(pages) {}
};

#endif
```

For this one, I made a simple Job class to represent a single print job. It just holds three things, an id, the userName of whoever sent it, and numPages for how many pages the job has. I gave it a default constructor that sets everything to empty values, and a second constructor that only takes in the user's name and page count, since I let the Printer class handle assigning the actual id automatically instead of setting it manually every time.

Printer.h

```h
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <iomanip>
#include "job.h"

// class Printer: manages a queue of Jobs using a linked list (no STL)
class Printer {
    private:
        // node struct used internally to build the queue
        struct JobNode {
            Job data;
            JobNode* next;
        };

        JobNode* front;
        JobNode* back;

        int trackingID;   // auto increments every time a job is added
        int jobsLeft;      // how many jobs are still pending

        // small helper just to print a divider line
        void printDivider(char symbol = '-'){
            std::cout << std::string(50, symbol) << std::endl;
        }

    public:
        // constructor
        Printer() : front(nullptr), back(nullptr), trackingID(0), jobsLeft(0) {
            printDivider('=');
            std::cout << " PRINTER READY " << std::endl;
            printDivider('=');
        }

        // destructor, clears any remaining nodes so nothing leaks
        ~Printer(){
            while(front != nullptr){
                JobNode* temp = front;
                front = front->next;
                delete temp;
            }
            printDivider('=');
            std::cout << " PRINTER SHUT DOWN " << std::endl;
            printDivider('=');
        }

        // enqueue: add a new job to the back of the queue, ID is auto assigned
        void addJob(Job newJob){
            newJob.id = ++trackingID;

            JobNode* newNode = new JobNode;
            newNode->data = newJob;
            newNode->next = nullptr;

            if(front == nullptr){
                front = newNode;
                back = newNode;
            } else {
                back->next = newNode;
                back = newNode;
            }
            jobsLeft++;

            std::cout << "[QUEUED] Job #" << newJob.id << "  |  User: " << std::left << std::setw(10) << newJob.userName
                      << "|  Pages: " << newJob.numPages << std::endl;
        }

```

```
69          // dequeue: process the job currently at the front (first come, first served)
70          void processNextJob(){
71              if(front == nullptr){
72                  std::cout << ">> No jobs left to print, printer is idle." << std::endl;
73                  return;
74              }
75
76              JobNode* temp = front;
77              printDivider();
78              std::cout << " PRINTING JOB #" << temp->data.id << std::endl;
79              std::cout << "   User : " << temp->data.userName << std::endl;
80              std::cout << "   Pages: " << temp->data.numPages << std::endl;
81              printDivider();
82
83              front = front->next;
84              if(front == nullptr){
85                  back = nullptr;
86              }
87
88              delete temp;
89              jobsLeft--;
90              std::cout << ">> Job completed.\n" << std::endl;
91          }
92
93          // process every pending job in order
94          void processAllJobs(){
95              if(front == nullptr){
96                  std::cout << ">> No jobs in queue." << std::endl;
97                  return;
98              }
99              while(front != nullptr){
100                 processNextJob();
101             }
102         }
103
104         // check if there are no pending jobs
105         bool isEmpty(){
106             return front == nullptr;
107         }
108
109         // print a quick status line, how many jobs are waiting
110         void showStatus(){
111             printDivider('*');
112             std::cout << " STATUS: " << jobsLeft << " job(s) currently in queue" << std::endl;
113             printDivider('*');
114         }
115 };
116
117 #endif
```

For this one, I built the Printer class to manage the print queue using my own linked list, no STL. It uses a private JobNode struct with front and back pointers, a trackingID counter for auto assigning IDs, and a jobsLeft counter to track pending jobs. addJob() assigns the next ID and adds the job to the back of the queue while increasing jobsLeft, and processNextJob() checks if the queue is empty first, then removes and prints the job at the front while decreasing jobsLeft. processAllJobs() loops through everything left, isEmpty() checks if the queue has nothing in it, and showStatus() shows how many jobs remain. I also added constructor and destructor messages for startup and shutdown, with the destructor cleaning up any leftover nodes to avoid memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.